Zen-Guard-Stream: A Streaming Safety Classifier Built on Qwen3Guard-Stream-0.6B

Technical Whitepaper v2025.05

Zach Kelling
Zen LM Research Team
research@zenlm.org
papers.zenlm.org

May 2025

Abstract

Zen-Guard-Stream is a streaming safety classifier built on Alibaba's **Qwen3Guard-Stream-0.6B** [1], a purpose-built token-level guardrail model. It is *not* a from-scratch model and uses no bespoke architecture: the base is `Qwen/Qwen3Guard-Stream-0.6B`, a safety variant of Qwen3-0.6B (a dense decoder-only transformer, `Qwen3ForCausalLM`; ≈0.6B parameters, 28 layers, hidden size 1024, GQA with 16 query / 8 key–value heads, head dimension 128, vocab 151,936) carrying two lightweight classification heads attached to the final transformer layer. Unlike a general-purpose LLM, Qwen3Guard-Stream is engineered for exactly this job: it consumes a response token by token *as it is generated* and emits a safety verdict at each step over three severity tiers — **safe**, **controversial**, and **unsafe** — across 119 languages and dialects [1], so that an unsafe continuation can be intercepted before a complete harmful output is delivered, which post-hoc filtering cannot do without buffering. **License note (the reason for this rebase):** the previous base, Qwen2.5-3B, was released under the non-commercial *Qwen Research License*, which barred commercial deployment of a derivative without a separate license from Alibaba. Rebasing onto Qwen3Guard-Stream-0.6B — which Alibaba ships under **Apache-2.0** — resolves that license issue: the streaming guard is now fully open for commercial use. We report no *Zen-measured* benchmark numbers; the fabricated figures (e.g. "ToxiGen 98.7%," "1.7 ms overhead," "0.8% FPR") and the "1.5B-parameter" size in earlier revisions have been removed/corrected.

Contents

1 Introduction **3**
1.1 Model Overview . . . 3

2 Architecture **4**
2.1 Streaming Safety Formulation . . . 4
2.2 Token-Level Classifier Architecture . . . 4
2.3 Intervention Protocol . . . 4
2.4 Context Window . . . 5

3 Training Methodology **5**
3.1 Approach . . . 5

4 Evaluation **5**
4.1 Recommended Evaluation Before Deployment . . . 6

5 Integration Guide **6**
5.1 Hook Interface 6
5.2 Deployment Topologies 6

6 Related Work **6**

7 Conclusion **7**

1 Introduction

The deployment of streaming generative AI creates a fundamental safety tension: streaming delivers tokens to users immediately as they are generated, enabling low-latency conversational experiences, but this means harmful content reaches users before a post-hoc safety filter can intervene.

Traditional mitigation approaches are inadequate for streaming contexts:

1. **Post-hoc filtering**: Buffer the complete output, apply safety filter, deliver (or withhold) the result. This works for batch generation but adds 100–500ms latency in streaming contexts, negating the latency advantage of streaming.
2. **Prompt-level filtering**: Classify the input prompt before generation begins. This misses unsafe content that emerges mid-generation in response to apparently benign prompts (indirect jailbreaks, gradual elicitation).
3. **Sampling-time constraints**: Apply logit masks to suppress individual tokens associated with harmful vocabulary. This misses multi-token harmful patterns and is trivially circumvented by paraphrasing.

Zen-Guard-Stream operates at the token generation level with access to the full generation context, enabling detection of harmful trajectories through multi-token patterns while maintaining streaming delivery of safe content.

1.1 Model Overview

Table 1: Zen-Guard-Stream specification. Base-model facts are those of the upstream Qwen3Guard-Stream-0.6B / Qwen3-0.6B [1]; Zen-Guard-Stream packages it for in-loop streaming intervention.

Parameter	**Value**
Base model	Qwen3Guard-Stream-0.6B (Alibaba)
License	Apache-2.0 (rebase from non-commercial Qwen2.5-3B)
Underlying base	Qwen3-0.6B, dense decoder-only (`Qwen3ForCausalLM`)
Total Parameters	≈0.6B
Layers / hidden size	28 / 1024
Attention heads (Q / KV, GQA)	16 / 8
Head dimension	128
Vocabulary	151,936
Classification heads	two token-level heads on the final transformer layer
Severity tiers	safe / controversial / unsafe
Languages	119 languages and dialects
Integration mode	Per-token streaming moderation alongside a generator
Version	v2025.05

Safety benchmark accuracies, false-positive rates, and per-step overheads measured by Zen are deliberately omitted (see abstract); the values in earlier revisions were not measured. The rebase from the non-commercial Qwen2.5-3B base onto the Apache-2.0 Qwen3Guard-Stream-0.6B removes the prior commercial-use restriction.

2 Architecture

2.1 Streaming Safety Formulation

Zen-Guard-Stream models streaming safety as a sequential decision problem. The upstream Qwen3Guard-Stream head emits a three-tier verdict $v_t \in \{\text{safe}, \text{controversial}, \text{unsafe}\}$ at each generation step [1]. For intervention we reduce this to a binary control signal: at each step t, given the prompt x and the generated sequence so far $y_{<t}$,

$$s_t = f_\theta(x, y_{<t}) \in \{0, 1\}, \tag{1}$$

where $s_t = 1$ (the *safe*, and optionally *controversial*, tiers) indicates a safe trajectory and $s_t = 0$ triggers an intervention. Because the upstream model attends over the full prompt-plus-response context at each step, the verdict reflects multi-token harmful patterns rather than isolated tokens.

2.2 Token-Level Classifier Architecture

Zen-Guard-Stream is the Qwen3Guard-Stream-0.6B token-level guardrail [1], run alongside the main generator:

- ≈0.6B parameters in a 28-layer causal transformer (hidden size 1024), the Qwen3-0.6B configuration.
- Grouped-query attention (16 query / 8 key–value heads), head dimension 128, vocabulary 151,936.
- *Two* lightweight classification heads attached to the final transformer layer: one scores the input prompt (nine categories, including jailbreak), the other scores the response as it is generated (eight categories) [1]. Each head emits a three-tier verdict (safe / controversial / unsafe) at every step rather than a single binary trajectory score.
- Designed to receive the response token by token and classify each new token instantly, so it can run interleaved with — or on a separate stream from — the main generator.

For the deployment in Section 2.1 we reduce the upstream per-step tier verdict to a binary safe/intervene signal $\hat{s}_t$ via a tier threshold; this is a packaging choice on top of the upstream head, not a change to the model. Running the small 0.6B classifier concurrently with the main generator limits the marginal latency impact, but we do not claim a specific per-step overhead, as it depends on the hardware, the generator size, and how much computation actually overlaps. The earlier revision's "1.5B, 24-layer, 100K-vocabulary, 4-KV-head" description did not match any deployed base and has been corrected to the actual Qwen3Guard-Stream-0.6B / Qwen3-0.6B configuration.

2.3 Intervention Protocol

When the classifier triggers ($\hat{s}_t < \tau_{\text{safe}} = 0.15$), the streaming intervention protocol executes:

1. **Immediate halt**: The unsafe token is suppressed; streaming delivery pauses.
2. **Context assessment**: The main model's logit distribution is inspected; if a safe alternative completion is available with $p > 0.3$, it is substituted and streaming resumes.
3. **Graceful degradation**: If no safe continuation is available, a stop message is inserted and the session ends gracefully with an explanation.

4. **Logging**: The full context at time of intervention is logged for safety audit review.

The substitution mechanism (step 2) is novel: rather than simply halting generation, Zen-Guard-Stream attempts to steer the generation onto a safe trajectory, improving user experience in borderline cases where the main model has accessible safe alternatives.

2.4 Context Window

The classifier sees the recent generation context (prompt plus generated tokens) within the base model's native context window [1], which is more than enough to capture multi-turn elicitation patterns in a typical conversation. The "learned context compression module" producing a $\text{Compress}_\phi(\cdot)$ summary embedding, described in earlier revisions, did not exist in the deployed model and has been removed; the model simply uses its native context window.

3 Training Methodology

3.1 Approach

The base, Qwen3Guard-Stream-0.6B, is *already* a token-level safety classifier: the Qwen team trained its streaming heads on over 1.19M human-annotated and synthetically generated safety samples (shared across the Qwen3Guard series) to emit per-step safe / controversial / unsafe verdicts during incremental generation [1]. The upstream streaming-safety capability therefore comes from Alibaba's training run, not from Zen. On top of it, the Zen packaging reduces the per-step tier verdict to a binary safe/intervene control signal, biased toward recall on genuinely harmful continuations,

$$\mathcal{L} = w_{\text{FN}}\mathcal{L}_{\text{FN}} + w_{\text{FP}}\mathcal{L}_{\text{FP}} + \lambda\mathcal{L}_{\text{calibration}}, \tag{2}$$

so that false negatives (missing unsafe trajectories) are penalized more heavily than false positives (blocking safe creative content).

We do not publish a Zen training corpus: the specific figures in earlier revisions (a 500M-trajectory corpus with per-source percentages, and a five-round "continuous adversarial training" curriculum) were fabricated and did not describe a real training run. The honest statements are that the base is Alibaba's Apache-2.0 Qwen3Guard-Stream-0.6B and that Zen adds threshold/intervention packaging; we make no quantitative accuracy claim without a rigorous, reproducible evaluation.

4 Evaluation

We report no *Zen-measured* benchmark numbers. The upstream Qwen3Guard technical report [1] does publish safety metrics for the Qwen3Guard series, and notes that even its smallest (0.6B) variants rival or exceed much larger guardrail models; but those headline F1 figures are reported for the *generative* 8B model, and we do not restate them here as if they were the streaming 0.6B model's or the Zen packaging's. We have not run a rigorous, reproducible streaming-safety evaluation of our own. The classification-accuracy, false-positive-rate, streaming-overhead, and multi-turn-jailbreak tables in earlier revisions (e.g. ToxiGen 98.7%, 0.8% FPR, 1.7 ms/step overhead, 90.3% multi-turn detection) were fabricated — they did not come from any measured evaluation — and have been removed rather than replaced with invented numbers.

The defensible *qualitative* argument for this design remains: a trajectory-level classifier inside the generation loop can, in principle, intervene before a complete harmful output is delivered, which post-hoc filtering cannot do without buffering, and it can use multi-token context that a per-token logit mask cannot. Whether a given fine-tune actually achieves high recall at an

acceptable false-positive rate and overhead is an empirical question that must be answered on the operator's own workload (Section 4.1).

4.1 Recommended Evaluation Before Deployment

Before relying on Zen-Guard-Stream, operators should measure, on their own traffic: detection recall on unsafe trajectories (including multi-turn elicitation), false-positive rate on benign creative/medical/security content, and the actual per-step latency overhead on their hardware and generator. We provide the model and the integration hook; the validation is the operator's responsibility.

5 Integration Guide

5.1 Hook Interface

Zen-Guard-Stream is integrated via a pre-token hook in the generation loop:

Listing 1: Integration Example

```
from zen_guard import StreamGuard

guard = StreamGuard.load("zen-guard-stream-v2025.05")

def generation_hook(context: list[int], next_token: int) -> int:
    safety_score = guard.score(context, next_token)
    if safety_score < 0.15:
        # Unsafe trajectory detected
        safe_alt = guard.get_safe_alternative(context)
        if safe_alt is not None:
            return safe_alt  # Substitute safe token
        else:
            return guard.STOP_TOKEN  # Graceful halt
    return next_token  # Pass through safe token
```

5.2 Deployment Topologies

Three topologies are possible; the right choice and its actual overhead depend on the operator's hardware and generator, which is why we give no overhead numbers (the figures in earlier revisions were not measured):

Table 2: Deployment configurations (qualitative).

Config	Hardware	Trade-off
Co-located (recommended)	Same GPU as generator	Lowest added latency; shares GPU
Sidecar (separate GPU)	Dedicated accelerator	Decoupled scaling; network hop
CPU sidecar	CPU	Cheapest; highest latency, limited concurrency

6 Related Work

Streaming content safety has been addressed through output filtering [2], circuit breaker mechanisms [3], and classifier guidance [4]. Token-level safety has been studied through vocabulary constraints [5] and watermarking [6]. Qwen3Guard [1] introduces a dedicated streaming

guardrail (Qwen3Guard-Stream) with token-level classification heads that emit a three-tier verdict during incremental generation. Zen-Guard-Stream applies that in-loop classifier within the generation loop, reducing its per-step tier verdict to a safe/intervene control signal; the model itself is the Apache-2.0 Qwen3Guard-Stream-0.6B [1].

7 Conclusion

Zen-Guard-Stream is a streaming safety classifier built on Alibaba's Qwen3Guard-Stream-0.6B [1] (itself a token-level safety variant of Qwen3-0.6B); it is not a from-scratch model and is ≈0.6B parameters, not the "1.5B" of earlier revisions — it is a redistribution of a purpose-built streaming guardrail with threshold/intervention packaging. Its premise is that a token-level classifier inside the generation loop can intercept an unsafe continuation before a complete harmful output is delivered, which post-hoc filtering cannot do without buffering. We deliberately make no *Zen-measured* benchmark or latency claims; the inflated accuracy/FPR/overhead figures of earlier revisions have been removed as fabrications, and the upstream report's headline F1 figures are for the generative 8B model and are not restated here as the 0.6B streaming model's. **License (the reason for this rebase):** the previous base, Qwen2.5-3B, was under the non-commercial Qwen Research License, which barred commercial use of a derivative; rebasing onto the **Apache-2.0** Qwen3Guard-Stream-0.6B resolves that — the streaming guard is now fully open for commercial deployment. Operators should validate the model on their own traffic before relying on it.

Attribution and License

The base weights, training data, and token-level safety capability are Alibaba's Qwen3Guard-Stream-0.6B (`Qwen/Qwen3Guard-Stream-0.6B`), itself a safety variant of Qwen3-0.6B; Zen contributes the threshold/intervention packaging. Qwen3Guard-Stream-0.6B is licensed under **Apache-2.0**, which is why this model was rebased off the non-commercial Qwen Research License-bound Qwen2.5-3B: commercial use is now unencumbered. We thank the Qwen team for releasing the base model openly under a permissive license.

References

[1] Qwen Team, Alibaba (2025). Qwen3Guard Technical Report. arXiv:2510.14276. Base model: `Qwen/Qwen3Guard-Stream-0.6B` (Apache-2.0), a token-level safety variant of Qwen3-0.6B.

[2] Lees, A. et al. (2022). A New Generation of Perspective API. arXiv:2202.11176.

[3] Zou, A. et al. (2024). Improving Alignment and Robustness with Circuit Breakers. arXiv:2406.04313.

[4] Dhariwal, P. & Nichol, A. (2021). Diffusion Models Beat GANs on Image Synthesis. NeurIPS 2021.

[5] Krause, B. et al. (2021). GeDi: Generative Discriminator Guided Sequence Generation. EMNLP 2021.

[6] Kirchenbauer, J. et al. (2023). A Watermark for Large Language Models. ICML 2023.